

RoPE：旋转位置编码

导论

Transformer 的自注意力机制能够并行建模序列中不同 Token 之间的依赖关系，但其计算本身不包含 Token 的先后顺序，因此必须额外引入位置信息。原始 Transformer 采用正弦—余弦函数构造绝对位置编码，随后相对位置表示、Transformer-XL 相对位置编码等方法进一步将 Token 之间的距离直接引入注意力计算。

旋转位置编码，英文为 Rotary Position Embedding，简称 RoPE，由 Su 等人提出。其基本思想是根据 Token 的绝对位置，对查询向量 Q 和键向量 K 的每组二维分量执行角度不同的旋转。对于位于位置 i 和 j 的查询向量与键向量，旋转后的内积满足： $\tilde{q}_i^T \tilde{k}_j = q_i^T R_{j-i} k_j$

其中， R_{j-i} 是由相对位置 $j-i$ 决定的旋转矩阵。该关系表明，RoPE 虽然根据绝对位置分别旋转 Q 和 K ，但最终的注意力分数能够自然包含 Token 之间的相对位置信息。与直接将位置向量加到 Token Embedding 上的方法相比，RoPE 将位置信息融入查询向量和键向量的几何方向，并且不需要额外的可训练位置参数。原文还讨论了 RoPE 对序列长度的适应性、相对距离建模以及在线性自注意力中引入相对位置信息的能力。

凭借结构简单、计算高效以及能够统一表示绝对位置和相对位置等特点，RoPE 已成为 Transformer 位置编码中的一种重要方法，并为后续长上下文位置缩放方法提供了基础。

Transformer 的自注意力机制本身不包含顺序信息。对于输入序列 $x_1, x_2, \dots, x_n$

如果仅使用自注意力，模型能够看到所有 Token，却无法直接判断某个 Token 位于第几个位置，也无法区分“奶龙喜欢小七”和“小七喜欢奶龙”，因此，Transformer 必须额外引入位置编码。

在旋转位置编码 RoPE 出现之前，主流位置编码大致经历了以下发展过程：

1. 可学习绝对位置编码；
2. 正弦—余弦绝对位置编码；
3. 相对位置表示；
4. Transformer-XL 相对位置编码；
5. 相对位置偏置；
6. 解耦位置编码；
7. 旋转位置编码 RoPE。

严格来说，位置编码的变体非常多，下面介绍的是 RoPE 之前最有代表性的主要方法。

设输入矩阵为 $X \in \mathbb{R}^{n \times d}$ ，其中 n 表示序列长度； d 表示隐藏向量维数。

自注意力首先计算 $Q = XW_Q$ ； $K = XW_K$ ； $V = XW_V$

然后计算 $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$ ，这里的注意力分数只取决于 Token 向量之间的相似性。假设对输入 Token 的顺序进行某种排列，自注意力的输出

也会按照同样的方式排列，但其计算规则本身不会发现 Token 的原始顺序发生了变化。这种性质称为**置换等变性**。

因此，模型需要额外获得 Token 的绝对位置；两个 Token 之间的相对距离；两个 Token 之间的相对距离。

一、可学习绝对位置编码

最直接的方法是为序列中的每个位置定义一个可训练向量。

设位置 p 的位置向量为 $e_p \in \mathbb{R}^d$ ，Token Embedding 为 $x_p \in \mathbb{R}^d$ ，模型输入为 $h_p = x_p + e_p$ ，其中 x_p 表示位置 p 上的 Token 内容； e_p 表示位置 p 对应的位置编码； h_p 表示同时包含内容和位置信息的输入向量。

例如，模型可以维护一个位置嵌入矩阵 $E_{\text{pos}} \in \mathbb{R}^{L_{\text{max}} \times d}$ ，其中 L_{max} 表示模型支持的最大序列长度。第 p 个位置直接查表得到 $e_p = E_{\text{pos}}[p]$

优点：

1. 实现简单；
2. 位置向量可以根据训练数据自动调整；
3. 不需要人为设计位置函数；
4. 在训练长度范围内通常具有较好的拟合能力。

局限性：

1. 最大长度固定

如果位置嵌入矩阵只包含 $0, 1, \dots, L_{\text{max}} - 1$ ，那么模型通常无法直接处理超过 L_{max} 的位置。

2. 长度外推能力较弱

位置 1000 和位置 1001 的向量是分别学习的，模型并不知道它们在数学上应当具有某种连续关系。

3. 相对位置不显式

模型可以知道一个 Token 位于位置 p ，但两个 Token 之间的距离 $j-i$ 需要模型通过训练间接推断。

BERT、早期 GPT 等模型采用过这种可学习的绝对位置编码。

二、正弦-余弦绝对位置编码

原始 Transformer 使用的是固定的正弦—余弦位置编码。

(<https://arxiv.org/pdf/1706.03762>)

对于位置 p 和维度编号 $2m, 2m+1$ ，定义

$$\text{PE}(p, 2m) = \sin\left(\frac{p}{10000^{2m/d}}\right)$$

$$\text{PE}(p, 2m+1) = \cos\left(\frac{p}{10000^{2m/d}}\right)$$

也可以定义频率 $\theta_m = 10000^{-2m/d}$ ，于是

$$\text{PE}(p, 2m) = \sin(p\theta_m); \text{PE}(p, 2m+1) = \cos(p\theta_m)$$

最终仍然把位置编码加到 Token Embedding 上: $h_p = x_p + \text{PE}(p)$

正弦和余弦函数具有良好的周期性和连续性。

对于固定的偏移量 r : $\sin((p+r)\theta_m)$ 和 $\cos((p+r)\theta_m)$, 可以由位置 p 对应的正弦、余弦通过线性组合表示。

例如:

$$\begin{aligned}\sin((p+r)\theta) &= \sin(p\theta)\cos(r\theta) + \cos(p\theta)\sin(r\theta) \\ \cos((p+r)\theta) &= \cos(p\theta)\cos(r\theta) - \sin(p\theta)\sin(r\theta)\end{aligned}$$

因此, 位置 $p+r$ 与位置 p 之间存在明确的数学关系。这使模型有可能通过线性运算学习相对位置

不同维度使用不同的频率 $\theta_0, \theta_1, \dots, \theta_{d/2-1}$, 其中高频分量对较小的位置变化更加敏感; 低频分量可以描述更长距离的位置变化。因此, 一个位置会被表示成多个不同尺度的周期信号。

优点:

1. 不需要训练额外的位置参数;
2. 可以计算任意位置的编码;
3. 不受固定位置嵌入表的严格限制;
4. 不同位置之间具有连续的数学关系。

局限性:

虽然正弦位置编码具有一定的相对位置性质, 但它仍然属于**绝对位置编码**。

模型接收到的是 $x_p + \text{PE}(p)$, 内容信息和位置信息直接相加, 模型仍需从混合后的向量中自行推断相对位置。

三、相对位置表示

绝对位置编码主要回答“一个 Token 位于哪里?”

相对位置编码则直接回答“两个 Token 相距多远?”

假设查询 Token 位于位置 i , 键 Token 位于位置 j , 相对位置为 $r=j-i$, 为每个相对距离定义一个向量 a_{j-i} , 传统注意力分数为 $e_{ij} = \frac{q_i^T k_j}{\sqrt{d}}$, 加入相对位置后, 可以写成

$$e_{ij} = \frac{q_i^T (k_j + a_{j-i}^K)}{\sqrt{d}}, \text{ 其中 } q_i \text{ 表示位置 } i \text{ 的查询向量; } k_j \text{ 表示位置 } j \text{ 的键向量; } a_{j-i}^K$$

表示距离 $j-i$ 对应的相对位置向量。

值向量也可以加入相对位置 $z_i = \sum_j \alpha_{ij} (v_j + a_{j-i}^V)$, 其中 α_{ij} 表示注意力权重; a_{j-i}^V 表示值向量使用的相对位置表示。

相对距离裁剪

序列可能很长, 如果为所有相对距离分别定义向量, 会产生大量参数。

因此通常设置最大相对距离 k : $\text{clip}(j-i, -k, k)$, 也就是说小于 $-k$ 的距离统一

视为 $-k$ ；大于 k 的距离统一视为 k 。

优点：

1. 直接表示两个 Token 之间的相对距离；
2. 更符合注意力机制比较 Token 对的计算方式；
3. 在处理局部依赖时更加自然；
4. 相同距离可以共享同一个位置表示。

局限性：

1. 每个 Token 对都需要处理相对位置；
2. 计算和存储成本可能增加；
3. 距离裁剪后，模型无法准确区分特别远的位置；
4. 实现比绝对位置编码更加复杂。

四、Transformer-XL 相对位置编码

Transformer-XL 对相对位置编码进行了进一步改进。它不仅考虑内容与内容之间的关系，还将内容与位置、全局内容偏置和全局位置偏置分别建模。

其注意力分数可以表示为 $e_{ij} = q_i^T k_j + q_i^T r_{i-j} + u^T k_j + v^T r_{i-j}$

其中 q_i 表示位置 i 的查询向量； k_j 表示位置 j 的键向量； r_{i-j} 表示相对位置 $i-j$ 的编码； u 表示全局内容偏置； v 表示全局位置偏置。

这四项分别表示：

1. 内容与内容的关系
 $q_i^T k_j$ 用于衡量两个 Token 内容是否相关。
2. 内容与相对位置的关系
 $q_i^T r_{i-j}$ 表示查询 Token 对某个相对距离的偏好。
3. 全局内容偏置
 $u^T k_j$ 表示模型整体对某类内容的偏好。
4. 全局位置偏置
 $v^T r_{i-j}$ 表示模型整体对某些相对距离的偏好。

优点：

Transformer-XL 的一个重要目标是支持分段处理长序列。

当模型缓存前一段序列的隐藏状态时，如果采用普通绝对位置编码，同一个缓存状态在不同片段中的位置可能产生冲突。

相对位置编码只关心 $i-j$ 因此更适合跨片段复用历史状态。

局限性：

1. 注意力分数分解较复杂；
2. 相对位置计算会增加实现难度；
3. 长上下文外推仍然受训练分布影响。

五、相对位置偏置

后来的一些模型不再把相对位置表示为完整向量，而是直接给注意力分数增加一个标量偏置。

注意力分数写成 $e_{ij} = \frac{q_i^T k_j}{\sqrt{d}} + b_{i-j}$ ，其中 b_{i-j} 表示相对距离 $i-j$ 对应的可学习标量偏置。

经过 Softmax 后 $\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_t \exp(e_{it})}$ ，如果某个相对距离对应的偏置较大，模型就更倾向于关注该距离上的 Token。

分桶相对位置偏置

如果为每个距离都学习一个偏置，长距离会产生大量参数。因此可以将距离划分为若干区间，也称为位置桶。例如距离 0 单独一个桶；距离 1 单独一个桶；距离 2 单独一个桶；较大的距离按照近似对数方式合并。

可以将分桶函数写成 $\text{bucket}(i-j)$ ，对应的偏置为 $b_{\text{bucket}(i-j)}$ ，较短距离使用精确表示，较长距离使用粗略表示。

T5 使用了这种相对位置偏置思想。

优点：

1. 结构简单；
2. 不需要对 Q、K 或 V 增加额外位置向量；
3. 直接作用于注意力分数；
4. 参数量较少；
5. 对长距离可以使用分桶压缩。

局限性：

1. 位置关系最终只由一个标量表示；
2. 表达能力弱于完整的相对位置向量；
3. 超出训练范围的位置通常会被映射到最后几个桶中；
4. 无法精确区分特别长的距离。

六、解耦位置编码

普通绝对位置编码直接将内容向量与位置向量相加 $h_i = x_i + p_i$ ，然后再生成查询和键。这种做法会将内容和位置混合在同一个向量中。

解耦位置编码将 Token 内容和位置分别建模。

设 c_i 表示位置 i 的内容向量； p_i 表示位置 i 的位置向量。注意力分数可以分解为多个部分 $e_{ij} = c_i^T W_{cc} c_j + c_i^T W_{cp} p_{i-j} + p_{i-j}^T W_{pc} c_j$

其中：

1. 内容到内容
 $c_i^T W_{cc} c_j$ 衡量两个 Token 内容之间的关系。
2. 内容到位置

$c_i^T W_{cp} p_{i-j}$ 表示查询内容与相对位置之间的关系。

3. 位置到内容

$p_{i-j}^T W_{pc} c_j$ 表示相对位置与键内容之间的关系。

DeBERTa 使用了类似的解耦注意力设计。

优点：

1. 内容信息和位置信息不再简单相加；
2. 可以分别建模内容—内容、内容—位置和位置—内容关系；
3. 位置关系表达更加细致。

局限性：

1. 计算结构更加复杂；
2. 需要额外的位置投影和参数；
3. 在大规模自回归模型中实现成本较高。

RoPE 出现之前的这些方法分别解决了不同问题，但也存在一些共同局限。

1. 内容和位置直接相加

可学习绝对位置编码和正弦位置编码都使用 $h_p = x_p + p_p$ ，这会让内容信息和位置信息混合在一起。

2. 相对位置计算较复杂

早期相对位置编码通常需要为每一对 Token 额外计算 a_{i-j} ，当序列长度为 n 时，一共有大约 n^2 个 Token 对。

3. 长上下文扩展困难

可学习位置编码受到最大位置表长度限制。相对位置偏置虽然可以分桶，但远离位置会失去精确性。

4. 注意力内积缺少统一的几何解释

早期方法通常通过向输入增加位置向量；向键和值增加相对位置向量；向注意力分数增加偏置来引入位置信息。

这些方法没有直接把位置关系统一地嵌入查询向量和键向量的几何结构中。旋转位置编码正是在这个背景下出现的。

七、旋转位置编码 RoPE

旋转位置编码的英文名称为 **Rotary Position Embedding**，简称 **RoPE**。

它的思想是根据 Token 的位置，对查询向量 Q 和键向量 K 的每一组二维分量进行旋转，使旋转后向量的内积自然依赖两个 Token 的相对位置。

RoPE 不再将一个完整的位置向量直接加到 Token Embedding 上，而是在计算注意力时，对 Q 和 K 进行位置相关的旋转。（<https://arxiv.org/pdf/2104.09864>）

RoPE 的二维旋转

考虑一个二维向量 $\begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$ ，将它旋转角度 ϕ ，得到 $\begin{pmatrix} x'_0 \\ x'_1 \end{pmatrix} = \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$ ，

展开后为 $x'_0 = x_0 \cos \phi - x_1 \sin \phi$ ； $x'_1 = x_0 \sin \phi + x_1 \cos \phi$ ，旋转不会改变向量的长度 $(x'_0)^2 + (x'_1)^2 = x_0^2 + x_1^2$ ，因此，RoPE 通过改变向量方向注入位置信息，而不会直接改变向量的欧几里得长度。

设每个注意力头的查询向量维数为偶数 d ： $q = (q_0, q_1, q_2, q_3, \dots, q_{d-2}, q_{d-1})$

RoPE 将相邻维度组成二维分量： (q_0, q_1) ； (q_2, q_3) ；...； (q_{d-2}, q_{d-1}) ，共有 $\frac{d}{2}$ 个二维分量，键向量 k 也采用相同的分组方法。

对于第 m 个二维分量，定义频率 $\theta_m = 10000^{-2m/d}$ ，其中 $m = 0, 1, \dots, \frac{d}{2} - 1$

如果 Token 位于位置 p ，那么第 m 个二维分量的旋转角度为 $\phi_{p,m} = p\theta_m$ ，因此，位置越靠后，累计旋转角度越大。不同维度对应不同的频率较高频率对短距离位置变化更敏感；较低频率适合表示较长距离的位置变化。

对于查询向量的第 m 组分量 (q_{2m}, q_{2m+1}) 在位置 p 上旋转后得到：

$$\tilde{q}_{p,2m} = q_{p,2m} \cos(p\theta_m) - q_{p,2m+1} \sin(p\theta_m)$$

$$\tilde{q}_{p,2m+1} = q_{p,2m} \sin(p\theta_m) + q_{p,2m+1} \cos(p\theta_m)$$

键向量使用完全相同的变换

$$\tilde{k}_{p,2m} = k_{p,2m} \cos(p\theta_m) - k_{p,2m+1} \sin(p\theta_m)$$

$$\tilde{k}_{p,2m+1} = k_{p,2m} \sin(p\theta_m) + k_{p,2m+1} \cos(p\theta_m)$$

对于位置 p ，定义旋转矩阵 $R_p = \begin{pmatrix} R(p\theta_0) & & & \\ & R(p\theta_1) & & \\ & & \ddots & \\ & & & R(p\theta_{d/2-1}) \end{pmatrix}$ ，其中

$$R(p\theta_m) = \begin{pmatrix} \cos(p\theta_m) & -\sin(p\theta_m) \\ \sin(p\theta_m) & \cos(p\theta_m) \end{pmatrix}$$

于是，位置 p 上的查询向量和键向量变为 $\tilde{q}_p = R_p q_p$ ； $\tilde{k}_p = R_p k_p$ ，这里 q_p 和 k_p 表示原始查询向量和键向量； R_p 表示位置 p 对应的旋转矩阵； $\tilde{q}_p$ 和 $\tilde{k}_p$ 表示加入位置信息后的向量。

假设查询 Token 位于位置 i ，键 Token 位于位置 j 。旋转后的注意力内积为 $\tilde{q}_i^T \tilde{k}_j$ ，代入旋转公式 $\tilde{q}_i^T \tilde{k}_j = (R_i q_i)^T (R_j k_j)$

根据矩阵转置规则 $\tilde{q}_i^T \tilde{k}_j = q_i^T R_i^T R_j k_j$ ，二维旋转矩阵满足 $R_i^T = R_{-i}$ ，同时满足

$R_{-i}R_j = R_{j-i}$, 因此 $\tilde{q}_i^T \tilde{k}_j = q_i^T R_{j-i} k_j$, 最终的位置部分只依赖 $j-i$, 也就是两个 Token 的相对位置。

这说明每个向量根据自己的绝对位置 i 或 j 进行旋转; 两个旋转向量的内积最终只与位置差 $j-i$ 有关。

举例:

假设每向后移动一个位置, 旋转角度增加 30°

原始查询向量和键向量都为 $q=(1,0)$; $k=(1,0)$

查询 Token 位于位置 2, 其旋转角度为 $2 \times 30^\circ = 60^\circ$

旋转后的查询向量为 $\tilde{q} = (\cos 60^\circ, \sin 60^\circ)$, 即 $\tilde{q} = \left(\frac{1}{2}, \frac{\sqrt{3}}{2}\right)$

键 Token 位于位置 5, 其旋转角度为 $5 \times 30^\circ = 150^\circ$

旋转后的键向量为 $\tilde{k} = (\cos 150^\circ, \sin 150^\circ)$, 即 $\tilde{k} = \left(-\frac{\sqrt{3}}{2}, \frac{1}{2}\right)$

两者内积为 $\tilde{q}^T \tilde{k} = \cos(150^\circ - 60^\circ)$, 所以 $\tilde{q}^T \tilde{k} = \cos 90^\circ = 0$

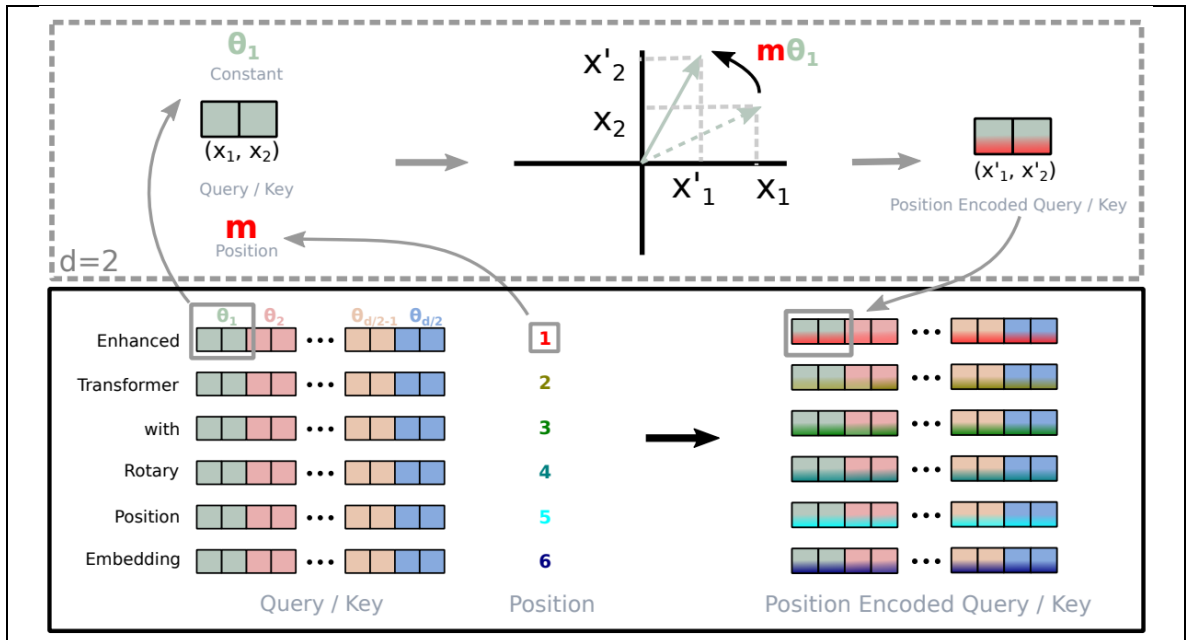
这里真正影响内积的是位置差 $5-2=3$, 对应的相对旋转角度为 $3 \times 30^\circ = 90^\circ$

Transformer 首先根据隐藏状态 X 计算 $Q = XW_Q$; $K = XW_K$; $V = XW_V$

然后对 Q 和 K 使用 RoPE: $\tilde{Q} = \text{RoPE}(Q)$; $\tilde{K} = \text{RoPE}(K)$

最后计算注意力 $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{\tilde{Q} \tilde{K}^T}{\sqrt{d}}\right)V$

通常只对 Q 和 K 进行旋转, 而不对 V 进行旋转。原因在于 Q 和 K 决定模型应该关注哪些 Token; 位置信息需要影响 Token 之间的匹配关系; V 主要负责携带最终被聚合的内容信息。



优点:

1. 自然表示相对位置

RoPE 最重要的性质是 $\tilde{q}_i^T \tilde{k}_j = q_i^T R_{j-i} k_j$, 注意力内积中的位置关系由 $j-i$ 决定。

2. 不需要额外可训练参数

原始 RoPE 的频率和旋转角度由固定公式计算, 不需要学习位置嵌入表。

3. 不直接混合内容与位置

RoPE 不使用 $x_p + p_p$, 而是在 Q 和 K 上进行正交旋转。

4. 保持向量长度

由于旋转矩阵是正交矩阵 $R_p^T R_p = I$, 因此 $|R_p q_p|_2 = |q_p|_2$, 位置编码主要改变向量方向, 而不会直接放大或缩小向量。

5. 适合 KV Cache

在自回归生成中, 已经计算出的键向量可以连同其位置信息一起存入 KV Cache。生成新 Token 时, 只需对新的 Q 和 K 使用当前位置的 RoPE。

局限性:

RoPE 虽然可以计算任意位置, 但这不意味着模型能够无限长度外推。当推理长度显著超过训练长度时, 可能出现高频维度旋转过快; 相位出现大量周期重复; 测试阶段的角度组合超出训练分布; 远距离注意力模式发生异常; 模型难以正确理解超长上下文。

因此, 长上下文模型通常会进一步使用位置插值; RoPE 线性缩放; NTK 感知缩放; 动态 NTK 缩放; YaRN 等频率缩放方法。这些方法的共同目标是让训练长度之外的位置旋转变化更加平缓。